Binary Trees and Binary Search Trees (BST)

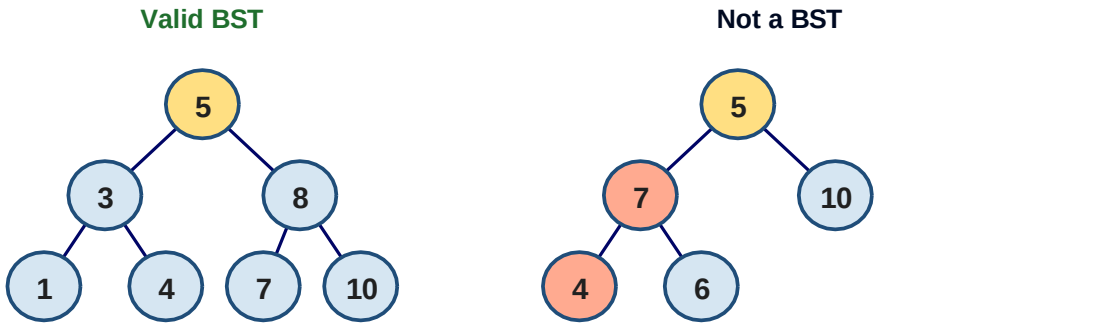
1. What is a binary search tree (BST)

It's a special binary tree that stores its values in a way that makes searching very fast.

For every node with value v:

- every value in the left sub-tree is < v
- every value in the right sub-tree is > v

This property must hold for every node, not just the root:



7 is greater than 5 but sits to the LEFT of it → violates the BST property

2. BST Implementation

We start with a Tree class that holds a reference to its root node:

```
# Node structure of each node of the binary tree.
class Tree:
    def __init__(self):
        self.root_node = None
```

a) Finding the Minimum and Maximum

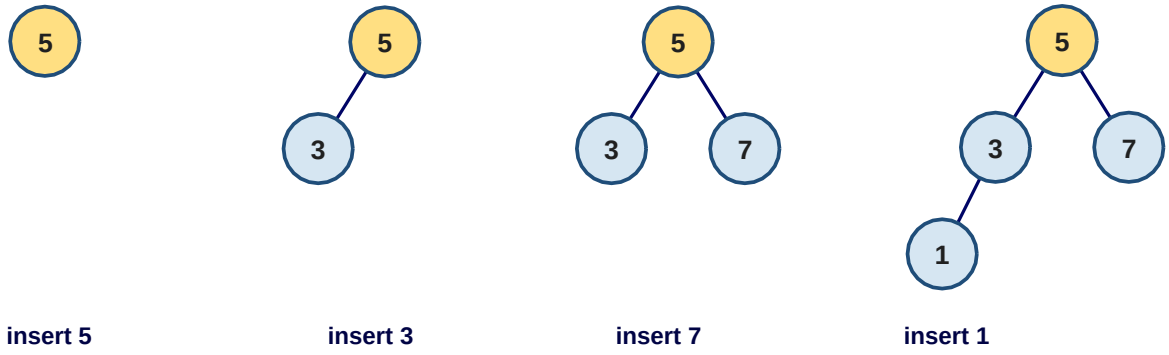
Because of the BST property, the smallest value lives at the leftmost node, and the largest at the rightmost node. We just keep walking left (or right) until there is no further child:

```
def find_min(self):
      current = self.root_node
      while current.left_child:
            current = current.left_child
      return current
```

Both run in O(h) where h is the height of the tree.

b) Inserting Nodes

To insert a value, compare it with the current node. Go left if it is smaller, go right if it is greater (or equal). Continue until we find an empty slot to attach the new node.



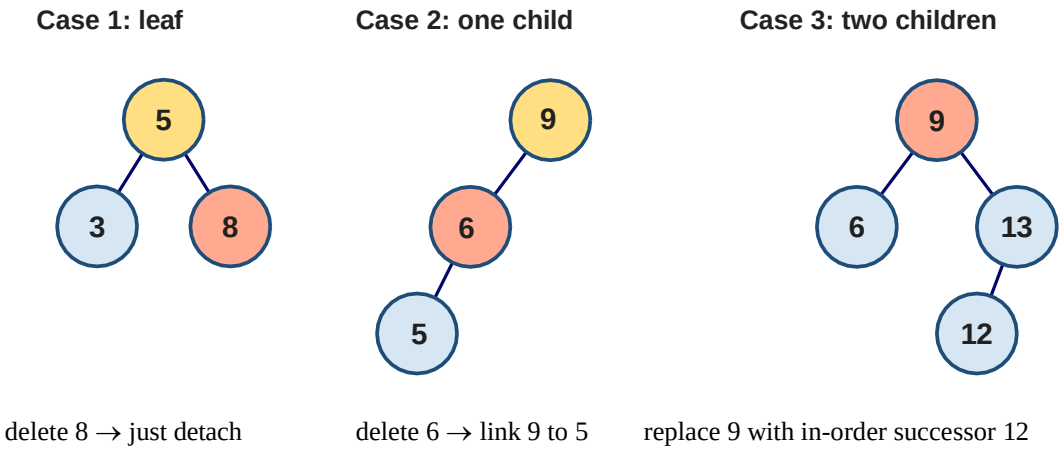
Building a BST by inserting 5, 3, 7, 1 in that order.

```
def insert(self, data):
	node = Node(data)
	if self.root_node is None:
		self.root_node = node
	else:
		current = self.root_node
		parent = None
	while True:
		parent = current
		if node.data < current.data:
			current = current.left_child
			if current is None:
				parent.left_child = node
				return
		else:
			current = current.right_child
			if current is None:
				parent.right_child = node
				return
```

Insertion runs in O(h).

c) Deleting Nodes — the Three Cases

Deletion is more involved than insertion. There are three scenarios depending on how many children the node we want to delete has:



The three deletion cases for a BST.

Case 1 — no children: Just detach the node from its parent.

Case 2 — one child: Rewire the parent of the deleted node directly to the deleted node's only child.

Case 3 — two children: Find the in-order successor (the smallest value in the right sub-tree), copy its value into the node we want to delete, then delete the in-order successor — which is always an easier case.

Helper that returns a node together with its parent:

```
def get_node_with_parent(self, data):
      parent = None
      current = self.root_node
      if current is None:
            return (parent, None)
      while True:
            if current.data == data:
                  return (parent, current)
            elif current.data > data:
                  parent = current
                  current = current.left_child
            else:
```

```
                parent = current
                current = current.right_child
                return (parent, current)
```

And the remove method using all three cases:

```
def remove(self, data):
      parent, node = self.get_node_with_parent(data)
      if parent is None and node is None:
            return False
      children_count = 0
      if node.left_child and node.right_child:
            children_count = 2
      elif (node.left_child is None) and (node.right_child is None):
            children_count = 0
      else:
            children_count = 1

      # Case 1: no children
      if children_count == 0:
            if parent:
                  if parent.right_child is node:
                        parent.right_child = None
                  else:
                        parent.left_child = None
            else:
                  self.root_node = None

      # Case 2: one child
      elif children_count == 1:
            if node.left_child:
                  next_node = node.left_child
            else:
                  node.right_child
            if parent:
                  if parent.left_child is node:
                        parent.left_child = next_node
                  else:
                        parent.right_child = next_node
            else:
                  self.root_node = next_node
```

```
        # Case 3: two children -> in-order successor
        else:
                parent_of_leftmost_node = node
                leftmost_node = node.right_child
                while leftmost_node.left_child:
                        parent_of_leftmost_node = leftmost_node
                        leftmost_node = leftmost_node.left_child
                node.data = leftmost_node.data
                if parent_of_leftmost_node.left_child == leftmost_node:
                        parent_of_leftmost_node.left_child =
                        leftmost_node.right_child
                else:
                        parent_of_leftmost_node.right_child =
                        leftmost_node.right_child
```

Removal runs in O(h).

d) Searching the Tree

Searching follows the same comparison rule as insertion: go left if the target is smaller, right if larger, return the node if we match, or return None if we fall off the tree:

```
def search(self, data):
        current = self.root_node
        while True:
                if current is None:
                        return None
                elif current.data is data:
                        return data
                elif current.data > data:
                        current = current.left_child
                else:
                        current = current.right_child
```

Quick test:

```
tree = Tree()
tree.insert(5)
tree.insert(2)
tree.insert(7)
tree.insert(9)
tree.insert(1)
for i in range(1, 10):
        found = tree.search(i)
        print('{}: {}'.format(i, found))
```

3. Tree Traversal

Visiting all nodes in a tree can be done in two main ways: **depth-first** and **breadth-first**.

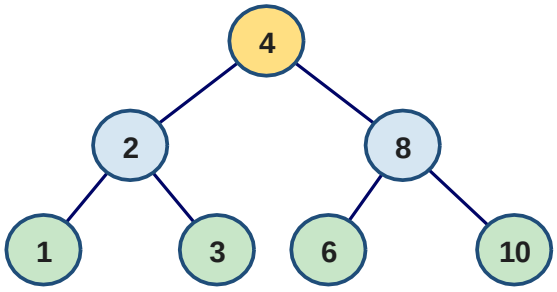
a) Depth-First Traversal

In depth-first traversal we follow a branch all the way down before backing up to try another branch. There are three flavours, depending on when we visit the parent node relative to its children:

in-order: $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 6 \rightarrow 8 \rightarrow 10$

pre-order: $4 \rightarrow 2 \rightarrow 1 \rightarrow 3 \rightarrow 8 \rightarrow 6 \rightarrow 10$

post-order: $1 \rightarrow 3 \rightarrow 2 \rightarrow 6 \rightarrow 10 \rightarrow 8 \rightarrow 4$



The same tree visited three different ways.

- In-order (left, node, right) — produces sorted output for a BST

```
def inorder(self, root_node):
        current = root_node
        if current is None:
                return
        self.inorder(current.left_child)
        print(current.data)
        self.inorder(current.right_child)
```

- Pre-order (node, left, right) — useful for copying a tree

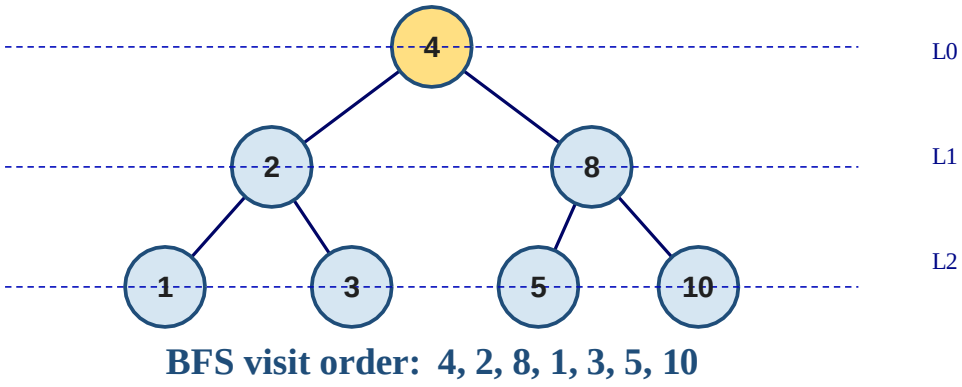
```
def preorder(self, root_node):
        current = root_node
        if current is None:
                return
        print(current.data)
        self.preorder(current.left_child)
        self.preorder(current.right_child)
```

- Post-order (left, right, node) — useful for deleting a tree

```
def postorder(self, root_node):
        current = root_node
        if current is None:
                return
        self.postorder(current.left_child)
        self.postorder(current.right_child)
        print(current.data)
```

b) Breadth-First Traversal

Breadth-first traversal visits all nodes on one level before moving to the next. It uses a queue: enqueue the root, then repeatedly dequeue a node, visit it, and enqueue its children.



BFS visits the tree level by level.

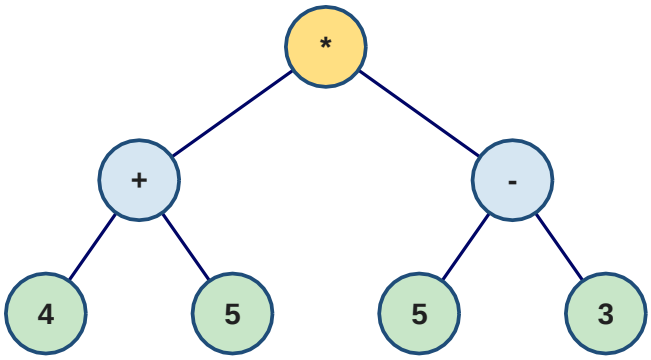
```
from collections import deque
class Tree:
	def breadth_first_traversal(self):
		list_of_nodes = []
		traversal_queue = deque([self.root_node])
		while len(traversal_queue) > 0:
			node = traversal_queue.popleft()
			list_of_nodes.append(node.data)
			if node.left_child:
				traversal_queue.append(node.left_child)
			if node.right_child:
				traversal_queue.append(node.right_child)
		return list_of_nodes
```

Why Use a BST?

Searching for a value in an unsorted list takes O(n). In a balanced BST it takes only O(log n) because each comparison cuts the search space in half. However, if you insert values in sorted order, the BST degenerates into a tilted line — essentially a linked list — and the advantage disappears.

Represent Arithmetic Expression in Trees

Trees can also represent arithmetic expressions. The operator goes at the root and the two operands hang underneath as children. For (4 + 5) * (5 - 3):



(4 + 5) * (5 - 3) = 9 * 2 = 18

An expression tree for (4 + 5) * (5 - 3).

The following code parses a reverse Polish expression using a stack and turns it into an expression tree:

```
class TreeNode:
      def  init (self, data=None):
            self.data = data
            self.right = None
            self.left = None

            expr = "4 5 + 5 3 - *".split()
            stack = Stack()

            for term in expr:
                  if term in "+-*/":
                        node = TreeNode(term)
                        node.right = stack.pop()
                        node.left = stack.pop()
                  else:
                        node = TreeNode(int(term))
                        stack.push(node)
```

Once built, the tree can be evaluated recursively:

```
def calc(node):
      if node.data == "+":
            return calc(node.left) + calc(node.right)
      elif node.data == "-":
            return calc(node.left) - calc(node.right)
      elif node.data == "*":
            return calc(node.left) * calc(node.right)
      elif node.data == "/":
            return calc(node.left) / calc(node.right)
      else:
            return node.data
      root = stack.pop()
      print(calc(root))  # 18 for (4 + 5) * (5 - 3)
```

Homework

Q1. Build a BST by inserting the following keys, in this order: 8, 3, 10, 1, 6, 14, 4, 7, 13. Draw the resulting tree.

Q2. Using your tree from Question 1, write the in-order, pre-order, and post-order traversal of the tree.

Q3. Implement (in Python) the find_max(self) method of the BST class, mirroring the find_min example given in the lecture.

Q4. Trace by hand the deletion of node 3 from the tree in Question 1. State which case applies and draw the tree afterwards.

Q5. Write a recursive Python function tree_height(node) that returns the height of a binary tree given its root.

Q6. Write a recursive Python function count_leaves(node) that returns the number of leaf nodes in a binary tree.

Q7. Use the RPN expression-tree code shown in section 8 to evaluate the expression 7 2 3 * - 5 + by hand. Show the stack contents after each token, then give the final result.

Q8. Trace the breadth-first traversal of the tree from Question 1 by hand: list the order in which nodes are visited.

Q9. Bonus: Modify the BST insert method so that duplicate keys are rejected instead of placed in the left sub-tree.